Technical Report on the Fruitfly Monitoring Platform

Prepared from the current implementation of the Fruitfly Monitoring Platform codebase

Date: 2026-04-03

Project	Fruitfly Monitoring Platform
Frontend	React, TypeScript, Vite
Backend	Node.js, Express, MySQL
Report Scope	Architecture, methodology, data flow, security, reporting, and implementation observations

This document presents the technical structure and operating methodology of the Fruitfly Monitoring Platform as implemented in the reviewed codebase.

Date: 2026-04-03

1. Executive Summary

The Fruitfly Monitoring Platform is a full-stack web application for agricultural pest surveillance. Its main goal is to collect, organize, visualize, and report fruit fly monitoring data coming from field devices. The platform combines user management, device registration, environmental sensing, fruit fly count recording, image capture, telemetry monitoring, geospatial visualization, and report generation into a single system.

Technically, the project follows a centralized IoT monitoring methodology:

1. Gateways and sensors are registered and assigned to farm users.
2. Sensor-linked devices submit environmental, count, image, and telemetry data to backend APIs.
3. The backend validates and stores observations in a relational MySQL database.
4. The frontend presents the data through dashboards, maps, sensor detail pages, image review views, and telemetry pages.
5. Users analyze recent records and generate reports for operational and pest-management decisions.

The application is implemented with a React and TypeScript frontend and a Node.js, Express, and MySQL backend.

2. Project Purpose

The system is designed to digitize fruit fly monitoring and reduce dependence on manual record keeping. Instead of relying only on isolated field checks, the platform creates a continuous monitoring environment in which authorized users can:

- manage gateways and sensors
- review environmental conditions such as temperature and humidity
- monitor fruit fly counts over time
- inspect captured images
- assess device condition through telemetry
- generate summary and analytics reports

This makes the project both a monitoring platform and a decision-support system.

3. Technology Stack

Frontend

- React 19
- TypeScript

- Vite
- Tailwind CSS
- React Router
- Leaflet and React Leaflet
- Recharts
- Axios

Reference:

- `client/package.json`

Backend

- Node.js
- Express 5
- MySQL2
- JWT authentication
- bcryptjs for password hashing
- Helmet for security headers
- express-rate-limit for API throttling
- jsPDF for PDF report generation

Reference:

- `server/package.json`

4. System Architecture

The project follows a client-server architecture with a relational database backend.

4.1 Frontend Layer

The frontend is responsible for:

- user authentication and protected route handling
- dashboard presentation
- map-based visualization of gateways and sensors
- sensor-level monitoring pages
- image analysis and review interfaces
- telemetry visualization
- report generation and download workflows

The route structure shows the major functional modules:

- landing page
- dashboard
- map
- sensors/data access
- gateways

- reports
- system telemetry
- fruitfly images
- admin pages
- profile

Reference:

- `client/src/App.tsx`

4.2 Backend Layer

The backend exposes REST endpoints under `/api` and is responsible for:

- authentication
- farm and device management
- ingestion of monitoring data from devices
- access control
- query aggregation
- report generation
- audit logging

The server initializes the database schema on startup, configures security middleware, mounts API routes, and serves the built frontend.

Reference:

- `server/src/server.js`

4.3 Database Layer

The platform uses MySQL with a normalized schema. The database separates user identity, device registration, time-series monitoring records, images, telemetry, reports, and audit logs.

Reference:

- `server/database/schema.sql`

5. Technical Methodology

The project methodology can be described as a field-data-to-decision pipeline.

5.1 Device Registration Methodology

The platform assumes a hierarchy:

- a gateway is the parent communication node
- one or more sensors are attached to a gateway

Gateways are created first, then sensors are created under a selected gateway. Each device has:

- a unique serial number

- a physical location
- latitude and longitude coordinates
- ownership linked to a user account
- an operating status

The serial number strategy is deterministic:

- gateways use a generated format like `eFF-G-001`
- sensors derive their serial number from the gateway, such as `eFF-G-001-N-001`

This methodology ensures traceable device identity and a clear monitoring topology.

References:

- `server/src/controllers/gateway.controller.js`
- `server/src/controllers/sensor.controller.js`

5.2 Data Ingestion Methodology

The backend accepts four main categories of field data:

1. Environmental readings
2. Fruit fly counts
3. Fruit fly images
4. System telemetry

For each incoming submission, the system resolves the submitted serial number into the corresponding registered sensor or gateway. If the device is not registered, the payload is rejected. This provides a simple device-validation mechanism.

Environmental Monitoring

Environmental readings store:

- temperature
- humidity
- time taken

These records are written into `environmental_readings`.

Reference:

- `server/src/controllers/environmentalData.controller.js`

Fruit Fly Count Monitoring

Count submissions store:

- fruitfly count
- time taken

These records are written into `fruitfly_counts`.

Reference:

- `server/src/controllers/countsData.controller.js`

Image Capture Methodology

Image uploads follow a slightly different pattern:

- the backend receives base64 image data
- the image is decoded and written to `uploaded_images`
- metadata is stored in `fruitfly_images`
- the image is linked back to a corresponding fruit fly count row through `image_id`

This creates a dual-record methodology:

- numerical record in `fruitfly_counts`
- visual evidence in `fruitfly_images`

Reference:

- `server/src/controllers/imageData.controller.js`

Telemetry Monitoring

Telemetry records represent device health and operational state. The telemetry controller accepts payloads for either a gateway or a sensor and stores:

- voltage
- current
- power
- signal strength
- CPU temperature
- timestamp

The implementation derives current from power or power from current when only one is provided, using a reference voltage if necessary. This reflects a fault-tolerant ingestion methodology for imperfect device payloads.

Reference:

- `server/src/controllers/systemTelemetry.controller.js`

5.3 Data Organization Methodology

The relational schema organizes the monitoring domain into the following main entities:

- `users`
- `gateways`
- `sensors`
- `environmental_readings`

- `fruitfly_images`
- `fruitfly_counts`
- `system_telemetry`
- `reports`
- `audit_logs`
- `content_blocks`

This schema supports:

- one-to-many ownership from users to gateways and sensors
- one-to-many relationships from sensors to readings, counts, and images
- optional image linkage from a fruit fly count to a captured image
- telemetry records for either gateways or sensors

This is a structured monitoring methodology rather than a flat log store.

Reference:

- `server/database/schema.sql`

5.4 Access-Control Methodology

The system uses JWT-based authentication and role-based authorization.

The main roles are:

- `admin`
- `manager`
- `viewer`

The access methodology is:

- authenticated users receive JWTs
- protected frontend routes require an authenticated user
- backend endpoints verify the bearer token
- viewer accounts are read-only
- manager and admin roles can perform writable farm operations

There is also a farm-ownership rule in which viewer accounts inherit access through `manager_user_id`, meaning a viewer can inspect a manager's farm data without becoming the owner of that data.

References:

- `client/src/contexts/AuthContext.tsx`
- `server/src/middleware/auth.middleware.js`
- `server/src/services/access.service.js`

5.5 Monitoring and Visualization Methodology

The frontend monitoring methodology is built around progressive visibility.

Dashboard

The dashboard loads location data in GeoJSON form and summarizes:

- total sensors
- active sensors
- inactive sensors
- gateway count
- total pest counts
- monitored areas

It also previews the latest telemetry row and latest fruit fly image. This creates a quick operational overview.

Reference:

- `client/src/pages/Dashboard.tsx`
- `server/src/controllers/dashboardData.controller.js`

Sensor Detail Monitoring

The sensor page lets a user:

- select a gateway
- choose a sensor
- fetch the latest environmental and fruit fly records
- view recent records as tables and charts
- auto-refresh data at an interval

This supports short-term operational monitoring and recent-history inspection.

Reference:

- `client/src/pages/Sensors.tsx`
- `server/src/controllers/data.controller.js`

Geospatial Monitoring

The map page loads gateway and sensor coordinates, overlays them on Leaflet maps, and enriches markers with the latest environmental and fruit fly readings. Sensor markers also reflect activity state visually.

This methodology supports spatial interpretation of pest pressure and device coverage.

Reference:

- `client/src/pages/MapView.tsx`
- `server/src/controllers/dashboardData.controller.js`

Image Review Workflow

The fruit fly image page implements a review lifecycle:

- fetch image rows

- filter by sensor and analysis status
- inspect images in a viewer
- mark analysis as `pending`, `analyzed`, or `failed`
- add reviewer notes
- optionally delete images

This introduces a human verification layer into the monitoring process.

Reference:

- `client/src/pages/FruitflyImages.tsx`
- `server/src/controllers/imageData.controller.js`

Telemetry Review Workflow

The telemetry page provides filters for:

- all devices
- sensors only
- gateways only

It calculates and visualizes averages and trends for voltage, current, power, and CPU temperature. This methodology extends the system beyond pest records into device reliability management.

Reference:

- `client/src/pages/SystemTelemetry.tsx`
- `server/src/controllers/systemTelemetry.controller.js`

5.6 Reporting Methodology

The reporting module aggregates monitoring data over user-selected date ranges.

Two report types are implemented:

- `summary`
- `analytics`

The backend report methodology includes:

- retrieving all sensors and gateways for a user
- aggregating environmental metrics such as average, min, and max values
- aggregating fruit fly totals, averages, and maxima
- counting active and inactive devices
- selecting top sensors by fruit fly totals for summary reports
- exporting results as PDF, CSV, and JSON

This is the project's main decision-support output.

References:

- `server/src/controllers/report.controller.js`
- `client/src/pages/Reports.tsx`

6. Data Flow

The technical data flow of the platform is:

1. User or device sends data to backend APIs.
2. Backend validates payload structure and resolves device serial numbers.
3. Backend stores records in MySQL tables.
4. Backend exposes read APIs that aggregate or filter the stored data.
5. Frontend fetches the data using Axios and displays it in pages, maps, charts, and tables.
6. Users review records and generate reports.

Operationally, the flow can be summarized as:

device input -> API ingestion -> database storage -> aggregation/query -> visualization -> reporting -> decision-making

7. Security and Reliability Features

The backend includes several platform protection measures:

- Helmet security headers
- CORS policy control
- request body size limits
- rate limiting for `/api`
- JWT verification
- role-based write restrictions
- audit logging for important actions

These are not advanced enterprise controls, but they provide a solid baseline for a monitoring platform of this scale.

References:

- `server/src/server.js`
- `server/src/middleware/auth.middleware.js`
- `server/database/schema.sql`

8. Strengths of the Current Technical Design

The project has several clear strengths:

- clean separation between frontend and backend concerns
- explicit relational modeling of the monitoring domain
- support for both numerical and visual monitoring evidence
- geospatial representation of monitoring coverage

- role-based access control
- reporting support for operational review
- telemetry support for device-health awareness
- a straightforward API structure that matches the domain

9. Technical Limitations and Observations

The current implementation is functional, but several technical limitations are visible from the code.

9.1 Public Device Ingestion Without Strong Device Authentication

The ingestion routes for environmental data, counts, images, and telemetry rely mainly on serial-number lookup. There is no device-level authentication token, signature, or certificate exchange visible in the current implementation. This means a valid serial number may be enough to submit data.

Relevant routes:

- `server/src/routes/environmentalData.routes.js`
- `server/src/routes/countsData.routes.js`
- `server/src/routes/imageData.routes.js`
- `server/src/routes/systemTelemetry.routes.js`

9.2 Limited Historical Analytics

The system focuses mostly on recent records and date-range summaries. There is no dedicated anomaly detection, trend forecasting, threshold alerting, or seasonal comparison logic.

9.3 Telemetry Constraint Not Fully Enforced in Schema

The schema notes that telemetry should ideally enforce exactly one of `gateway_id` or `sensor_id`, but this rule is handled at the application level rather than through a database constraint.

Reference:

- `server/database/schema.sql`

9.4 Reporting Scope Is Aggregative Rather Than Predictive

Reports summarize existing records well, but they do not yet appear to generate agronomic recommendations automatically or risk scores per site.

9.5 Sparse Top-Level Documentation

The implementation is stronger than the current repository-level documentation. The codebase contains a user-focused report, but technical architecture documentation is limited.

10. Recommendations

The following improvements would strengthen the project technically:

1. Add device authentication for ingestion endpoints.

2. Add threshold-based alerting for unusual fruit fly counts or telemetry failures.
3. Add database indexes for high-frequency query paths if data volume grows.
4. Enforce stronger telemetry constraints at the database level.
5. Introduce scheduled analytics such as weekly trend summaries.
6. Expand technical documentation for deployment, environment variables, API contracts, and database diagrams.
7. Add automated tests for controller logic and access-control rules.
8. Add image-processing or AI-assisted validation if automated pest recognition is a future goal.

11. Conclusion

The Fruitfly Monitoring Platform is a technically coherent agricultural monitoring system built around an IoT-style data lifecycle. Its methodology is based on registering devices, collecting time-stamped field observations, storing them in a structured relational model, presenting them through operational dashboards and maps, and generating reports for review and decision-making.

From a technical perspective, the project is best described as a web-based pest-monitoring and farm-device management platform with geospatial visualization, image review, telemetry tracking, and reporting capabilities. Its implementation is practical and domain-aligned, and it provides a strong foundation for future improvements in analytics, automation, and device security.

12. Key Source Files Reviewed

- `server/database/schema.sql`
- `server/src/server.js`
- `server/src/controllers/gateway.controller.js`
- `server/src/controllers/sensor.controller.js`
- `server/src/controllers/environmentalData.controller.js`
- `server/src/controllers/countsData.controller.js`
- `server/src/controllers/imageData.controller.js`
- `server/src/controllers/systemTelemetry.controller.js`
- `server/src/controllers/dashboardData.controller.js`
- `server/src/controllers/data.controller.js`
- `server/src/controllers/report.controller.js`
- `server/src/middleware/auth.middleware.js`
- `server/src/services/access.service.js`
- `client/src/App.tsx`
- `client/src/contexts/AuthContext.tsx`
- `client/src/pages/Dashboard.tsx`
- `client/src/pages/Sensors.tsx`
- `client/src/pages/MapView.tsx`

- `client/src/pages/SystemTelemetry.tsx`
- `client/src/pages/FruitflyImages.tsx`
- `client/src/pages/Reports.tsx`